



ZeroOTP

One idea, four pages

Guided, full-control, and passkey-only ways to protect the same TOTP codes.
Entirely in the browser.





Executive summary

- ZeroOTP is a suite of four pages, all generating the same standard TOTP codes entirely in the browser — no backend, ever.
- index.html is the entry point: a hub that links to the other three pages, lists known limitations, and summarizes how each method maps to NIST and Microsoft authentication frameworks.
- easy.html (guided) and full.html (everything) both offer a biometric passkey vault alongside Password Manager, Cookie, and LocalStorage.
- biometry.html is new: a dedicated, Chrome-only page for the passkey method alone, with an explicit browser check that stops the page from running anywhere else.
- Every method has its own icon, used consistently across all four pages so it's always clear which storage mechanism is in play.



Why a guided second factor

Authenticator apps ask a lot for a 30-second code

The old way

- Install a dedicated app
- Manually re-scan a QR code per device
- Hunt for the right code among many accounts
- Trust a third-party app with your secrets

The ZeroOTP way

- Open a browser tab — nothing to install
- Drag, drop, or paste a QR code once
- Pick the page — and method — that fits you
- Chrome's vault or a passkey holds the secret



Four pages, one idea

Pick the page that matches how much control you want

index.html

Hub / landing page. Links to the other three, plus limitations and the compliance summary.

easy.html

Guided default. Detects what's saved and shows only what's needed.



full.html

Every provisioning method and all four storage backends, on one page.



biometry.html

Dedicated passkey vault. Chrome only — the page stops itself elsewhere.





The four methods, at a glance

Same icon, same meaning, on every page



Password Manager

Chrome's built-in credential storage, via `navigator.credentials`.



Cookie

AES-256, Master-Password-encrypted, stored in a browser cookie.



LocalStorage

Same encryption as Cookie, different storage location.



Biometric

A passkey; the AES key is derived from the passkey itself. No Master Password exists for this method.



easy.html — bring in a code

Four QR-provisioning methods, in an even 2x2 matrix

- Drag a QR image off the page, or off your desktop
- Paste a copied QR image directly
- Drag an image in from another browser tab
- Drop a QR image file from Finder or Explorer
- Once decoded, choose how to protect it — see next slide

Drop a QR image

From this page or your desktop



Paste a QR image

Copy, then paste it in



Drop from another window

Drag from another tab



Drop a file

From Finder or Explorer





Protect it with...

The one deliberate choice in the guided flow

QR code
decoded



Password Manager



Biometrics

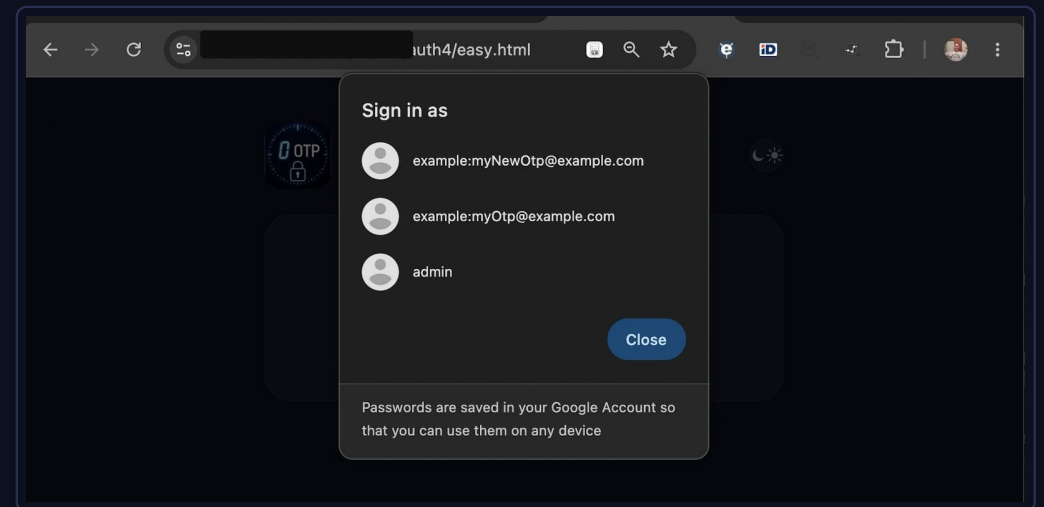
- Provisioning presents two protection options rather than saving to a single fixed backend
- Password Manager: Chrome's own native “Save password?” bubble does the rest
- Biometrics: enroll a passkey on the spot, protected by Touch ID / Windows Hello / Android biometrics
- Either way, the running code starts immediately — protecting it is a parallel step, not a blocker



Multiple accounts? Chrome asks.

The account picker is Chrome's own UI — nothing custom-built

- The native “Sign in as” chooser, same one used for saved website logins
- Appears automatically when more than one saved Password Manager entry matches
- A biometric vault instead shows a one-click “Unlock with passkey” prompt
- No custom picker to build or maintain for either path



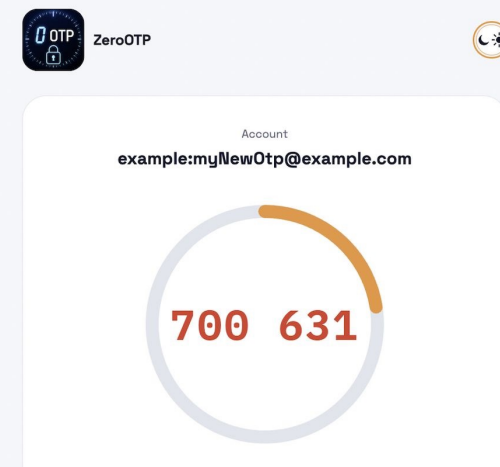
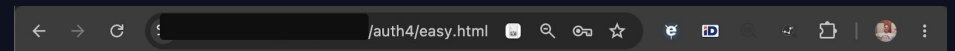


Your code, either theme

Dark by default; one click for light



Dark (default) — white digits



Light — red digits



full.html — every storage method, one page

Password Manager, Cookie, LocalStorage, and Biometric



Issuer + Secret + OTP only

- Once any one method is actively used (save or get), the other three collapse out of view
- A “Show all options” link brings the full picker back at any time
- “Save” is its own top-level section, matching the heading format used by Provisioning and Authenticator
- Issuer / Secret / OTP sit inside a double-bordered box, marking it as shared ground between the sections above and below
- The four provisioning methods are arranged as an even 2×2 matrix



biometry.html — passkey only, Chrome only

A dedicated page for one method, done properly



- Provision a code, then save or retrieve it with a passkey — nothing else on the page, four provisioning methods in an even 2×2 matrix
- Explicit about where the passkey lives: Chrome Password Manager, not this page's own storage
- Actively checks for Google Chrome on load, before wiring up any of its logic
- Once a secret is loaded, provisioning and the vault controls both hide — a “Manage vault” link brings them back when needed
- An expandable “What is PRF, and where does my data live?” section explains the mechanism in the same depth as this deck

Why Chrome-only, when WebAuthn is a cross-browser standard?

Because this page's messaging and testing are both anchored to Chrome's specific passkey storage and UI — not a claim about the underlying standard.



Consistent structure, across every page

Navigation and documentation follow the same pattern everywhere

- Every page includes a “← Back to ZeroOTP hub” link, so index.html is always one click away
- index.html closes with its Known Limitations section, placed after the NIST/Microsoft compliance summary
- The compliance mapping is available two ways: a styled page (compliance.html) for in-app reading, and a plain-text document (COMPLIANCE.md) as the source of truth
- Provisioning methods are laid out as a 2×2 matrix everywhere they appear — easy.html, full.html, and biometry.html alike



How the Biometric method actually works

PRF: a real secret, derived, never stored



- The credential-bound secret never leaves the authenticator chip — only the HMAC output does, and only after Touch ID / Windows Hello / Android biometrics succeed
- The AES key itself is never written anywhere — it's re-derived fresh on every unlock
- Fallback “gate” mode exists for authenticators without PRF support, but is meaningfully weaker: the key is stored (non-extractable) in IndexedDB, and the biometric check is an application-logic gate, not a cryptographic one



Security model, side by side

Four methods, four different guarantees

Method	Key derived from	Master Password?	Phishing-resistant?
Cookie	PBKDF2 (6,000 iter.)	Yes	No
LocalStorage	PBKDF2 (6,000 iter.)	Yes	No
Password Manager	Chrome's own encryption	No	No
Biometric (PRF)	Passkey PRF + HKDF	No	Yes

Biometric “gate” fallback mode (no PRF hardware) does not meet the phishing-resistant bar — see [js/biometric.js](#).



How this maps to NIST & Microsoft

A factual mapping, not a certification

Method	NIST SP 800-63B-4	Microsoft Entra strength
Cookie / LocalStorage / Password Manager (TOTP)	Software OATH token — AAL2, not phishing-resistant	MFA strength only
Biometric / passkey (PRF mode)	Syncable passkey — AAL2, genuinely phishing-resistant	Phishing-resistant MFA strength

- NIST SP 800-63B-4 (final, July 2025) introduced explicit guidance for syncable passkeys, integrating its 2024 interim guidance
- Chrome Password Manager stores syncable (not device-bound) passkeys — that caps this method at AAL2, not AAL3
- Full mapping, methodology, and source links: [COMPLIANCE.md](#)



Known limitations

Reported in full on [index.html](#)

- Low PBKDF2 iterations (Cookie / LocalStorage only): 6,000, intentionally low for demo speed
- Hardcoded salt (Cookie / LocalStorage only): a fixed string, not randomized per device
- No Master Password complexity check (Cookie / LocalStorage only): any string is accepted
- One entry per backend, with one exception: Cookie, LocalStorage, and Biometric each store exactly one Issuer/Secret pair; Chrome Password Manager manages multiple entries natively, with its own account picker



Annex

[full.html](#) in detail



full.html — every option, one page

Four storage backends

Backend	Protection
Chrome Password Manager	Chrome's own encryption
Cookie	AES-256, encrypted with a Master Password you choose
LocalStorage	AES-256, encrypted with a Master Password you choose
Biometric	Passkey-derived key (WebAuthn PRF) — no Master Password

- Four QR-provisioning methods, all in one page
- Once any one backend is used, the other three collapse out of view (see slide 10)



Thank you

- Repository: github.com/giovannicandotti/ZeroOTP
- Hub: [index.html](#) · Guided: [easy.html](#) · Full: [full.html](#) · Passkey only: [biometry.html](#)
- Compliance mapping: [compliance.html](#) (styled) or [COMPLIANCE.md](#) (source)
- License: MIT